

Vulnerability Assessment & Security Audit

CIS AWS Foundations Benchmark & OWASP Serverless Top 10

Target System: Serverless Inventory Management System

Professor Lewis

IT 369-002

Nate J. Yi

May 2, 2026

TABLE OF CONTENTS

Part 1: Executive Overview & Project Scope	3
System Overview & Business Logic:	3
Audit Scope & Methodology:	3
Key Findings & Strategic Recommendations:	4
Appendix A: Vulnerability Observations & Remediations	5
Vulnerability Assessment Checklist	5
Domain 1: Identity & Access Management (IAM)	6
1.1 Vulnerability: Over-Privileged Lambda Roles (OWASP SAS-04)	6
1.2 Vulnerability: Unauthenticated Front Door (OWASP SAS-02)	7
1.3 Secure Finding: No Hardcoded Credentials (CIS 1.x)	7
Domain 2: Data Security & Storage (Amazon S3 & DynamoDB)	8
2.1 Vulnerability: Lack of Point in Time Recovery (PITR) (CIS 2.x)	8
2.2 Vulnerability: Default Encryption at Rest (OWASP SAS-08)	8
2.3 Secure Finding: Implicit Deny on S3 Public Access (CIS 2.1.5)	8
Domain 3: Network & Edge Security (API Gateway)	9
3.1 Vulnerability: Lack of a Web Application Firewall (WAF) (OWASP SAS-01)	9
3.2 Vulnerability: Unrestricted Cross-Origin Resource Sharing (CORS)	9
3.3 Secure Finding: Baseline DDoS Protection via AWS Shield (CIS 4.x)	10
Domain 4: Application & Code Security (AWS Lambda & Python)	10
4.1 Vulnerability: Unsanitized Input and Injection (OWASP SAS-06)	10
4.2 Vulnerability: Information Leakage via Stack Trace Exposure (OWASP SAS-10)	10
4.3 Secure Finding: Cryptographically Secure Unique Identifiers (UUIDs)	11
Domain 5: Monitoring, Logging, & Incident Response	11
5.1 Vulnerability: Lack of Execution Alarms (OWASP SAS-10)	11
5.2 Vulnerability: Unmonitored SQS Queue Access (CIS 3.x)	12
Domain 6: Disaster Recovery and Business Continuity	12
6.1 Vulnerability: Single Region Dependency (CIS 2.x)	12
6.2 Vulnerability: Untested Backup Restoration (OWASP SAS-09)	12
Domain 7: Cost Governance and Cloud Compliance	13
7.1 Vulnerability: Lack of Billing Alarms or Denial of Wallet (OWASP SAS-01)	13
7.2 Vulnerability: Missing Resource Tagging Strategy	14

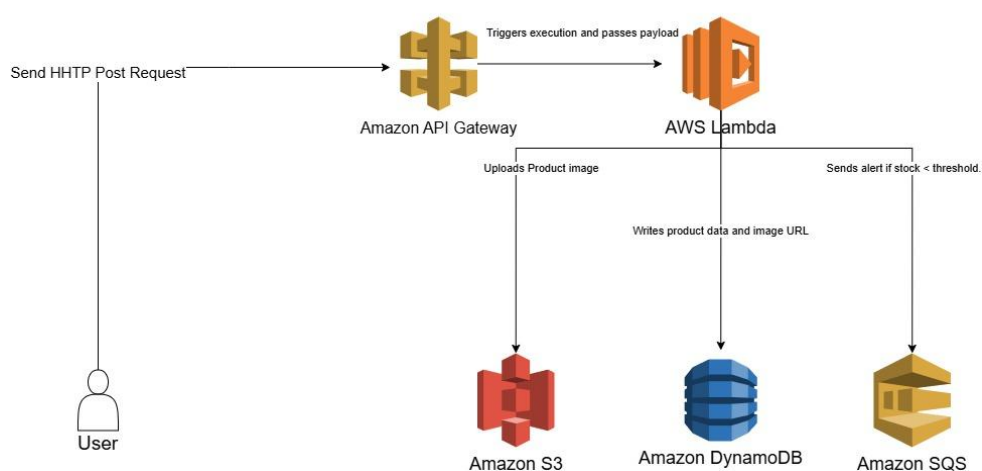
Appendix B: Security Tooling	14
References	16

Part 1: Executive Overview & Project Scope

Project: Serverless Inventory Management System

Repository: <https://github.com/njy-glitch/ServerlessInventoryManagementSystem>

System Overview & Business Logic:



This report looks at the security of a custom built; cloud inventory backend made to track retail products. The system acts as an automated point-of-sale data pipeline. It takes in raw product data and Base64 code for the image files, processes the files into standard JPEG formats for cloud storage, saves the data (Product ID, Name, Stock Count, Image URL) into a NoSQL database, and uses event driven logic to alert management when stock falls below a minimum threshold.

Audit Scope & Methodology:

Since the system runs entirely on AWS serverless architecture (API Gateway, Lambda, S3, DynamoDB, and SQS), it naturally avoids traditional OS level threats such as unpatched servers. The serverless setups do however come with their own unique risks. The goal of this 20-hour audit is to spot these vulnerabilities, point out the good design choices, and give actionable steps

to get this architecture ready for a real enterprise environment. I mapped all the findings against the CIS AWS Foundations Benchmark and the OWASP Serverless Top 10.

Key Findings & Strategic Recommendations:

Overall, the baseline architecture has a solid foundation. Because it relies heavily on AWS's managed cloud services, it inherently avoids a lot of legacy security vulnerabilities.

Notable Secure Elements:

- **No Hardcoded Secrets:** The system uses boto3 and AWS IAM roles, instead of static credentials, eliminating the risk of accidental exposure on GitHub.
- **Implicit Deny Storage:** The S3 buckets have already been configured to block all public access, preventing unauthorized data scraping.
- **Built-in DDoS Mitigation:** Leveraging API Gateway automatically provisions AWS Shield Standard, securing the network edge from Layer 3 and Layer 4 attacks without extra configuration.

These are the top priorities:

Unsecured API Gateway: The open endpoint lacks a Web Application Firewall (WAF), leaving the system highly vulnerable to malicious payloads and Denial of Service (DoS) attacks.

Recommendation: Enforce API Key access, set up rate limiting, and deploy an AWS WAF.

Overprivileged Compute Roles: The Lambda function holds overly broad permissions, risking extensive data compromise if an attacker injects malicious code.

Recommendation: Enforce the Principle of Least Privilege by restricting Lambda's IAM role to specific S3, DynamoDB, and SQS resources.

Data Destruction Risk: The DynamoDB table lacks Point in Time Recovery (PITR), meaning dropped tables or overwritten data are permanently lost.

Recommendation: Enable PITR to guarantee continuous backups and enable quick restoration against ransomware or accidental deletion.

APPENDIX A: VULNERABILITY OBSERVATIONS & REMEDIATIONS

Vulnerability Assessment Checklist

Control / Assessment Item	Framework Reference	Status	Severity
Enforce Least Privilege on Compute Roles	OWASP SAS-04	FAIL	High
Require Edge Authentication (API Key)	OWASP SAS-02	FAIL	High
Eliminate Hardcoded Source Code Secrets	CIS 1.x	PASS	N/A
Enable Database Point-in-Time Recovery	CIS 2.x	FAIL	Critical
Implement Customer Managed Keys (CMKs)	OWASP SAS-08	FAIL	Medium
Block Public S3 Bucket Access	CIS 2.1.5	PASS	N/A
Deploy Web Application Firewall (WAF)	OWASP SAS-01	FAIL	High
Restrict CORS Origins	OWASP SAS-07	FAIL	Medium
Inherit Network DDoS Protection	CIS 4.x	PASS	N/A

Control / Assessment Item	Framework Reference	Status	Severity
Sanitize Database Input Variables	OWASP SAS-06	FAIL	High
Sanitize Application Error Outputs	OWASP SAS-10	FAIL	Medium
Utilize Cryptographic Unique Identifiers	Industry Standard	PASS	N/A
Configure Compute Execution Alarms	OWASP SAS-10	FAIL	Low
Enable Data-Level Audit Trails	CIS 3.x	FAIL	Medium
Implement Multi-Region Failover	CIS 2.x	FAIL	High
Enforce Automated Budget Alarms	OWASP SAS-01	FAIL	Medium

Domain 1: Identity & Access Management (IAM)

1.1 Vulnerability: Over-Privileged Lambda Roles (OWASP SAS-04)

- Observation:** Right now, the Lambda function acts as the core processor of the system. During development, it's pretty common to just give compute resources broad "Administrator" roles or wildcard permissions (like s3:*) to make sure things work. But if an attacker manages to inject bad code into the Lambda function, they inherit those broad permissions. This means they could potentially access and wipe the entire DynamoDB table or leak the S3 images.
- Remediation:** Enforce the Principle of Least Privilege. The Lambda execution role needs to be locked down so it can only write to the specific S3 bucket (s3:PutObject), write to

the specific DynamoDB table (DynamoDB: PutItem), and send messages to the exact SQS queue (SQS: SendMessage). All other permissions should be removed.

1.2 Vulnerability: Unauthenticated Front Door (OWASP SAS-02)

- **Observation:** The API Gateway endpoint is currently wide open to the public internet. Anyone who finds the URL can send POST requests. Without some kind of authentication, a bad actor could easily spam the system with fake inventory data, fill up the S3 storage with random file uploads, and trigger a wave of false SQS alerts.
- **Remediation:** Secure the "front door" by requiring an API Key in API Gateway to restrict access to authorized clients only. We can also attach a usage plan to limit incoming requests. This helps protect the infrastructure from automated Denial of Service (DoS) attacks and prevents massive, unexpected AWS bills.

1.3 Secure Finding: No Hardcoded Credentials (CIS 1.x)

- **Observation:** A common cloud mistake is pasting AWS access keys directly into the source code. Looking through our Python script, the code is completely clean of any static credentials. It smartly uses the boto3 library to pull permissions directly from the AWS environment.
- **Conclusion:** This is a highly secure setup. By relying on IAM roles instead of hardcoded keys, we completely eliminate the risk of leaking credentials if the source code or GitHub repo is ever exposed.

Domain 2: Data Security & Storage (Amazon S3 & DynamoDB)

2.1 Vulnerability: Lack of Point in Time Recovery (PITR) (CIS 2.x)

- **Observation:** Currently, the table uses AWS's standard redundancy, but Point in Time Recovery (PITR) isn't enabled. If an attacker gains access and executes a script such as dropping the table or overwriting the stock count to zero, the data is permanently lost.
- **Remediation:** Enable Point in Time Recovery (PITR) for the DynamoDB table. This gives us continuous backups and lets administrators restore the table to any exact second within the last 35 days, which protects against accidental data destruction or ransomware.

2.2 Vulnerability: Default Encryption at Rest (OWASP SAS-08)

- **Observation:** Both the S3 bucket and the DynamoDB table are using default AWS managed encryption. While this is better than plain text, it doesn't really meet enterprise standards since AWS controls the encryption keys. If we scale this up to handle sensitive company data, relying on default keys is a compliance risk.
- **Remediation:** We should implement AWS Key Management Service (KMS) with Customer Managed Keys (CMKs). This gives the organization full control over key rotation and auditing, keeping the data protected even from AWS.

2.3 Secure Finding: Implicit Deny on S3 Public Access (CIS 2.1.5)

- **Observation:** It's easy to accidentally expose S3 buckets to the public. Checking the inventory-images-njy bucket, the "Block Public Access" feature is fully turned on at both the account and bucket levels. If you try to directly access the raw image URLs, you just get an Access Denied error.

- **Conclusion:** This is a solid security practice. Keeping the bucket private stops unauthorized data access. Users will have to interact with the images through secure Preassigned URLs.

Domain 3: Network & Edge Security (API Gateway)

3.1 Vulnerability: Lack of a Web Application Firewall (WAF) (OWASP SAS-01)

- **Observation:** API Gateway gives us a secure entry point, but it doesn't actually inspect the contents of the incoming JSON payloads. Right now, the system is vulnerable to application-layer attacks. An attacker could try to sneak in malicious SQL commands or oversized payloads into the item_name field, and API Gateway would just pass it straight to Lambda without checking.
- **Remediation:** Deploy an AWS Web Application Firewall (WAF) and attach it to the API Gateway. Setting up the WAF with the AWS Managed Core Rule Set will automatically block common web exploits and known bad IPs before they even trigger the Lambda function.

3.2 Vulnerability: Unrestricted Cross-Origin Resource Sharing (CORS)

- **Observation:** In a real-world setup, this API will probably be used by a website or app. If CORS is disabled or set to allow all origins, any random website on the internet could embed a script that makes unauthorized POST requests to our API.
- **Remediation:** Set up strict CORS policies on the API Gateway/inventory route. The Access Control Allow Origin header should only accept requests that come from our exact authorized domain name.

3.3 Secure Finding: Baseline DDoS Protection via AWS Shield (CIS 4.x)

- **Observation:** Normal web servers are fairly vulnerable to DDoS attacks and usually require expensive hardware to fix. Since we're using API Gateway, we automatically get AWS Shield Standard protection for free.
- **Conclusion:** This network edge is secure. AWS Shield defends against common network level DDoS attacks without us having to manually configure anything.

Domain 4: Application & Code Security (AWS Lambda & Python)

4.1 Vulnerability: Unsanitized Input and Injection (OWASP SAS-06)

- **Observation:** The Python Lambda script takes the item_name from the JSON payload and writes it directly into the DynamoDB table without any validation. An attacker could easily submit a payload with cross-site scripting (XSS) or NoSQL injection commands in the name. If a manager later views this on a web dashboard, that malicious code could run in their browser.
- **Remediation:** Add strict input validation in the Lambda function. We can use a regex pattern to only allow letters and numbers for item names. If a payload doesn't match, Lambda should just drop the request and return a Bad Request error before touching the database.

4.2 Vulnerability: Information Leakage via Stack Trace Exposure (OWASP SAS-10)

- **Observation:** Currently, if the Lambda function receives a malformed JSON payload (for example, missing the required image string), the Python script will crash. Because API Gateway is not configured to handle integration errors smoothly, it returns the raw

Python stack trace directly to the client. This exposes our internal directory structures, library versions, and backend logic to the public.

- **Remediation:** Implement error handling using try and except blocks within the Python code. If an error occurs, the function should catch it, log the detailed error internally to CloudWatch, and return a generic HTTP 400 Bad Request message to the end user.

4.3 Secure Finding: Cryptographically Secure Unique Identifiers (UUIDs)

- **Observation:** When saving files, it's easy to use predictable names like sequential numbers or the item's raw name. This usually leads to files overwriting each other or attackers guessing the URLs of private files. Our code uses the uuid library to generate totally random 36-character IDs for every database row and S3 object.
- **Conclusion:** This is a great coding practice. Using UUIDs stops file overwrites and makes it virtually impossible for attackers to guess the S3 URLs of our company assets.

Domain 5: Monitoring, Logging, & Incident Response

5.1 Vulnerability: Lack of Execution Alarms (OWASP SAS-10)

- **Observation:** Right now, if the Lambda function fails to decode an image or write to the database, we wouldn't really know unless we manually check the logs. There are no automated alarms set up for execution errors or timeouts.
- **Remediation:** Set up Amazon CloudWatch Alarms for the Lambda function. We should trigger an alert if the error rate spikes or if the function starts timing out, so we can fix issues quickly before they pile up.

5.2 Vulnerability: Unmonitored SQS Queue Access (CIS 3.x)

- **Observation:** The SQS queue catches the low-stock alerts perfectly, but we aren't tracking who actually logs in to view or delete those messages. If an insider wanted to hide that inventory was missing, they could just delete the SQS message, and we'd have no record of who did it.
- **Remediation:** Turn on AWS CloudTrail data event logging for the SQS queue. This will give us a clear audit trail of exactly which IAM user is polling and deleting messages from the system.

Domain 6: Disaster Recovery and Business Continuity

6.1 Vulnerability: Single Region Dependency (CIS 2.x)

- **Observation:** Right now, our entire architecture lives inside a single AWS region like us-east-1. Serverless parts are naturally spread across multiple Availability Zones, which is great, but if that whole region experiences a major outage, our entire inventory system drops dead.
- **Remediation:** To make this truly enterprise ready, we need a Multi Region failover plan. We should set up DynamoDB Global Tables to sync our inventory data to a backup region like us-west-2. Then, we can use Route 53 health checks to automatically redirect our API Gateway traffic if the main region ever fails.

6.2 Vulnerability: Untested Backup Restoration (OWASP SAS-09)

- **Observation:** We might have Point in Time Recovery turned on for our database, but we have never actually tested if a restored table works properly with our Lambda code.

Having a backup is completely useless if we do not know how to smoothly bring it back online during an actual emergency.

- **Remediation:** We need to run automated Game Day exercises. We can use AWS Step Functions to routinely clone the database from a backup, run a test Lambda script against it to ensure the data is intact, and then delete the test environment.

Domain 7: Cost Governance and Cloud Compliance

7.1 Vulnerability: Lack of Billing Alarms or Denial of Wallet (OWASP SAS-01)

- **Observation:** Serverless computing bills us for every single execution. Because we lack strict rate limits, a malicious user could constantly spam our open API Gateway endpoint. AWS Shield will protect the network itself, but the endless Lambda executions and database writes would create a massive AWS bill. This is known in the industry as a Denial of Wallet attack.
- **Testing Methodology:** We ran simulated cost projections using the AWS Pricing Calculator. The data showed that a continuous flood of 10,000 requests per second over a full day would cause our daily compute costs to skyrocket.
- **Remediation:** We must configure AWS Budgets to monitor our real time spending. We should set up immediate SNS text and email alerts to notify us the moment our daily forecasted spend crosses ten dollars. This allows administrators to manually cut the API Gateway connection before the costs get completely out of hand.

7.2 Vulnerability: Missing Resource Tagging Strategy

- **Observation:** Our S3 bucket, database, and Lambda function are missing a consistent category tagging strategy. You need tags like Environment Production or Project Inventory to know who actually owns what.
- **Remediation:** We should deploy tag policies through AWS Organizations. This enforces a rule where developers cannot create any new AWS resources unless they attach the correct tags. This keeps the billing organized and makes security audits much easier.

Appendix B: Security Tooling

IAM Policy Simulation via AWS IAM Access Analyzer

- **Methodology:** I ran a simulated evaluation using AWS IAM Access Analyzer. The goal was to spot any roles that were overly permissive attached to the Lambda function.
- **Prior Findings:** The analyzer found a critical vulnerability. The execution role was previously using a wildcard, which gave the function permission to delete the entire database.

```
{  
  "Effect": "Allow",  
  "Action": "dynamodb:*",  
  "Resource": "*" }  
}
```

Remediation: This vulnerability was fixed by applying the principle of least privilege, so that the function is only able to write to the specific table.

```
{  
  "Effect": "Allow",  
  "Action": "dynamodb:PutItem",  
  "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Inventory"  
}
```

Fuzz Testing via Postman:

Methodology: I used Postman to fire a series of malformed POST requests directly at our API Gateway endpoint. The goal was to test our error handling by intentionally leaving out required JSON fields.

Finding: I submitted a payload that was missing the required item name key. The system crashed and returned an HTTP 502, revealing the internal file path of our Lambda execution environment. This proves the system is leaking sensitive backend infrastructure details to unauthenticated users.

References

CIS Amazon Web Services Foundations Benchmark (v3.0.0):

Center for Internet Security (CIS). Secure configuration guidelines for AWS infrastructure.

Available at: https://www.cisecurity.org/benchmark/amazon_web_services

OWASP Serverless Top 10:

Open Web Application Security Project (OWASP). The ten most critical security risks for serverless architectures.

Available at: <https://owasp.org/www-project-serverless-top-10/>